

Memory

2026 Semester 1 SOFTENG 370: Operating Systems

Talia Xu

Lecture 14

1.0.0

Based on original slides by Professor Jon Eyolfson.

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

Using the Page Tables for Every Memory Access is Slow

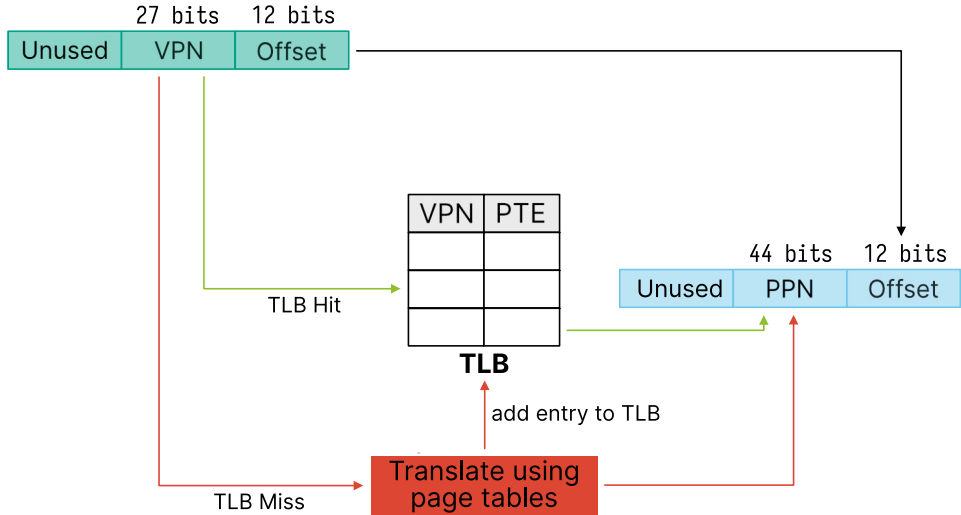
We need to follow pointers across multiple levels of page tables!

We'll likely access the same page multiple times
(close to the first access time)

A process may only need a few VPN $\rightarrow$ PPN mappings at a time

Our solution is another computer science classic: caching

A Translation Look-Aside Buffer (TLB) Caches PTEs



Effective Access Time (EAT)

Assume a single page table

(there's only one additional memory access in the page table)

$$\text{TLB_Hit_Time} = \text{TLB_Search} + \text{Mem}$$

$$\text{TLB_Miss_Time} = \text{TLB_Search} + 2 \times \text{Mem}$$

$$\text{EAT} = \alpha \times \text{TLB_Hit_Time} + (1 - \alpha) \times \text{TLB_Miss_Time}$$

If $\alpha = 0.8$, $\text{TLB_Search} = 10 \text{ ns}$, and accesses take 100 ns , calculate EAT

Context Switches Require Handling the TLB

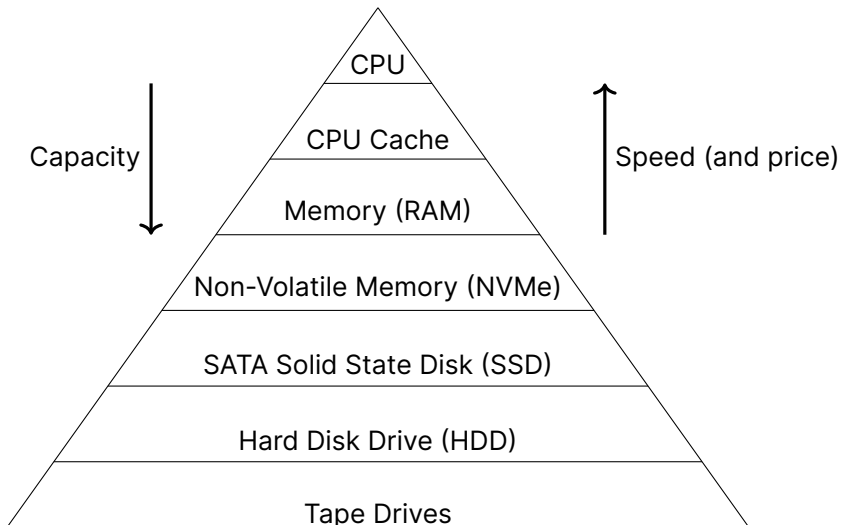
You can either flush the cache, or attach a process ID to the TLB

Most implementation just flush the TLB

RISC-V uses a `sfence.vma` instruction to flush the TLB

On x86 loading the base page table will also flush the TLB

Computer Memory Hierarchy is a Trade-off of Capacity and Speed



We Want to Hide the Hierarchy from the User

Each level wants to pretend it has the speed of the layer above it
and the capacity of the layer below it

The memory used by all the processes may exceed the amount of physical
memory

Not all of them may be in use at the same time

Only keep referenced pages in memory, put others on disk

Swap pages back to memory when they're needed

Demand Paging

We use memory as a cache for the file system

Map memory pages to file system blocks

Only load them into memory when they're used through the page fault handler

If the page doesn't represent a file, we can map it to swap space

Move it to disk if we need to use more physical memory

A Processes' Working Set Should Fit in Physical Memory

Given an amount of time, the number of pages your process uses is called its *working set*

If you cannot fit your working set into physical memory your process will *thrash*

It's constantly moving entries in and out of cache

Page Replacement Algorithms

Optimal

Replace the page that won't be used for the longest

Random

Replace a random page

First-in First-out (FIFO)

Replace the oldest page first

Least Recently Used (LRU)

Replace the page that hasn't been used in the longest time

Page Replacement Evaluation

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

We'll use this for a bunch of examples during this lecture

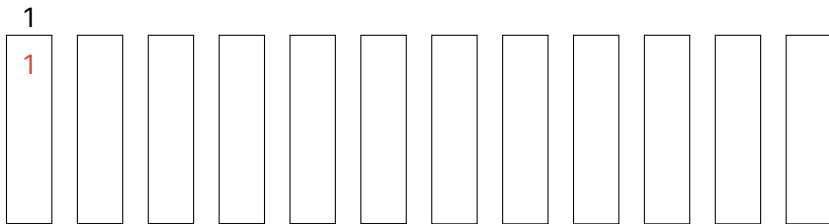
We want the fewest number of page faults

For every example we'll find the number of page faults

Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

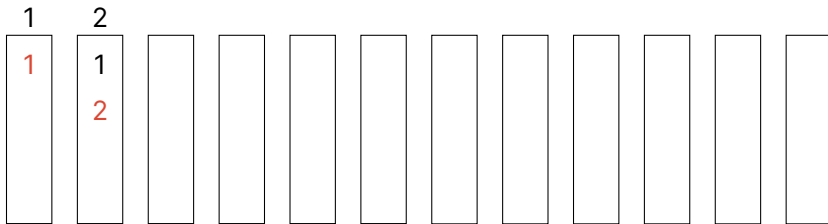
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

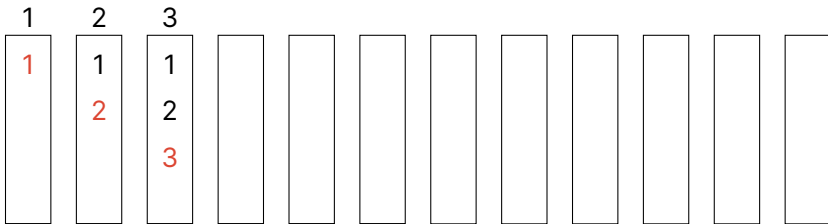
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

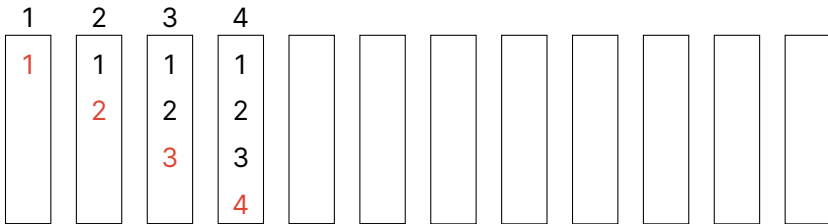
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

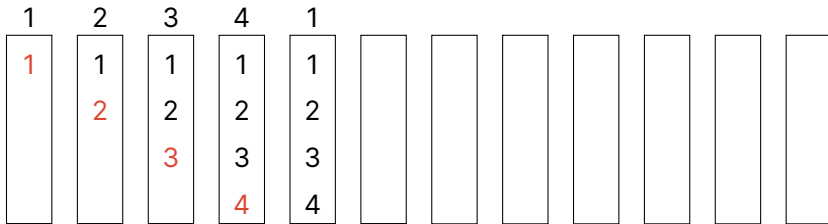
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

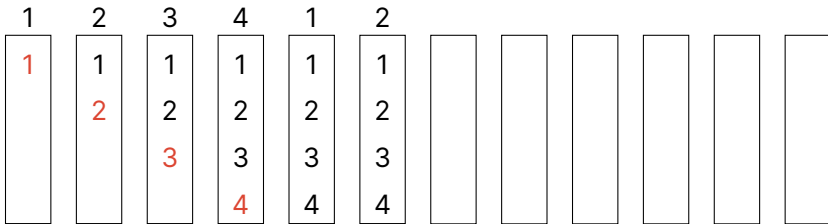
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

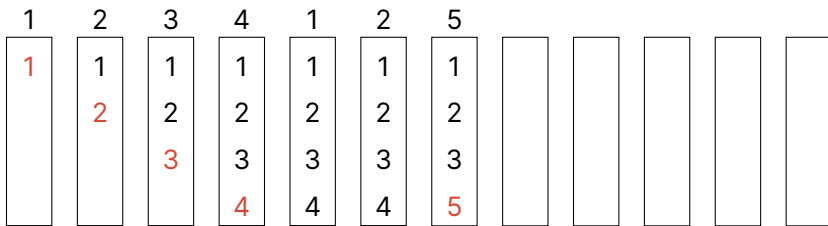
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

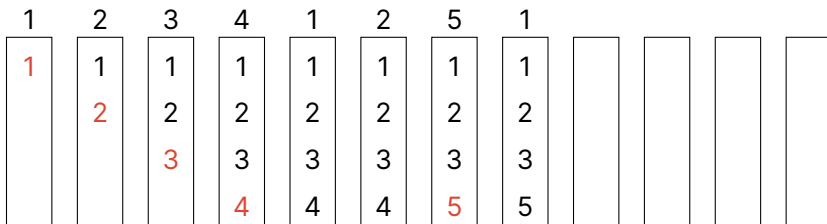
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

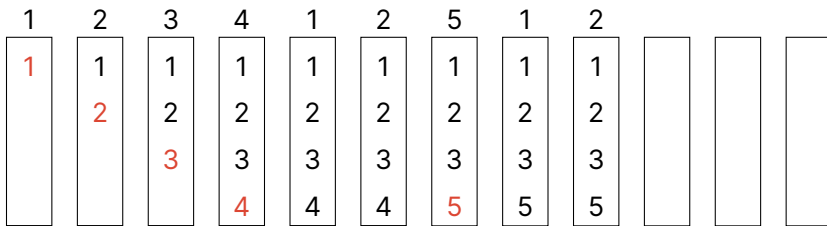
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

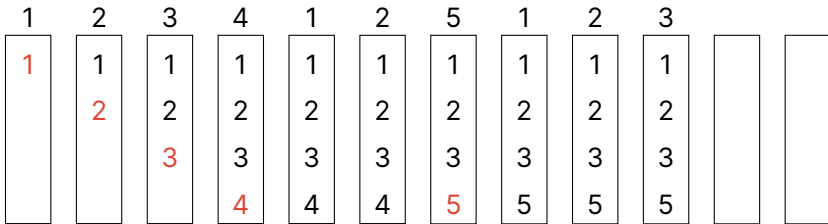
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	
1	1	1	1	1	1	1	1	1	1	4	
	2	2	2	2	2	2	2	2	2	2	
		3	3	3	3	3	3	3	3	3	
			4	4	4	5	5	5	5	5	

Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	4	4
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	3	3	3	3	3	3
			4	4	4	5	5	5	5	5	5

Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

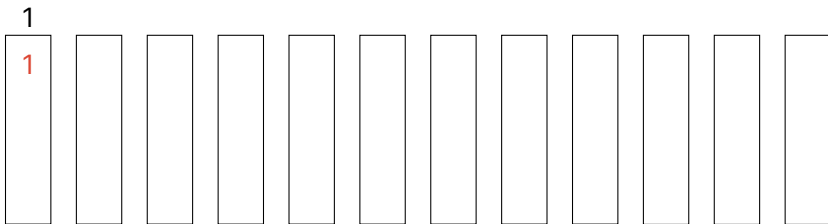
1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	4	4
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	3	3	3	3	3	3
			4	4	4	5	5	5	5	5	5

6 page faults

FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

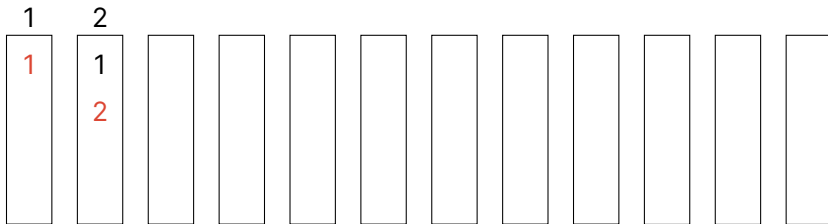
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

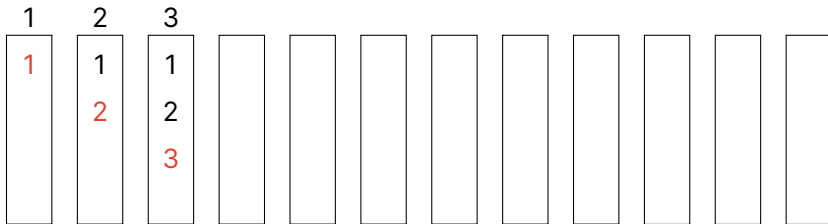
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

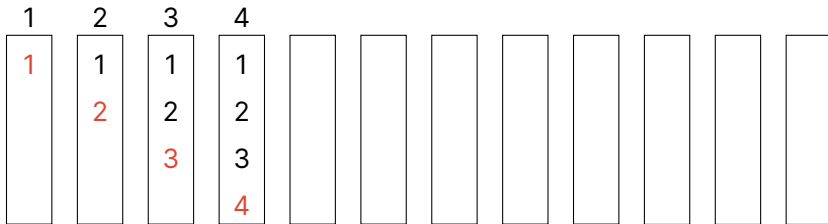
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

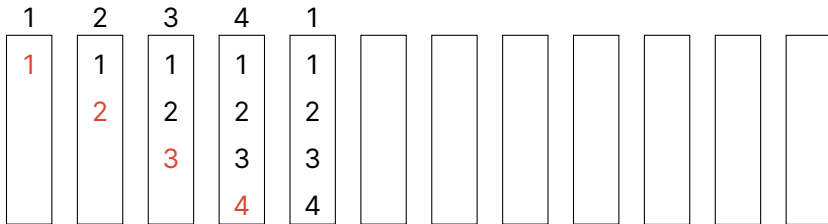
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

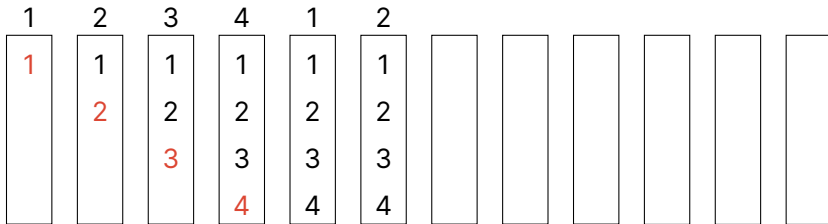
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

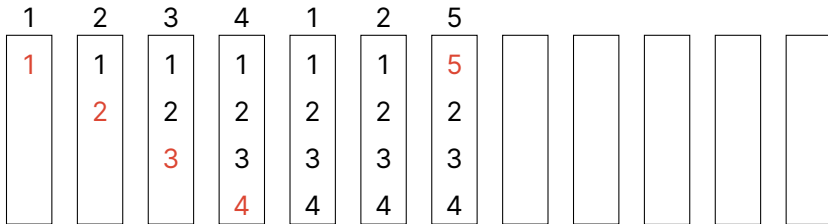
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

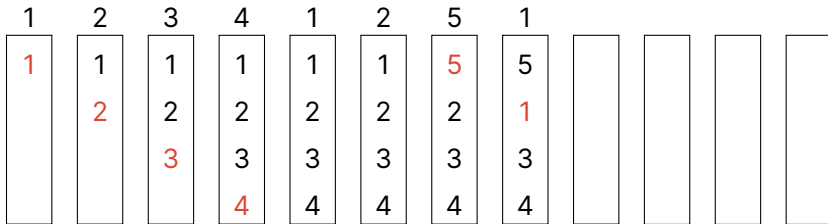
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

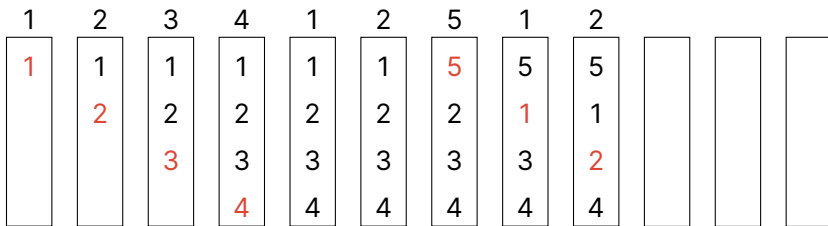
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

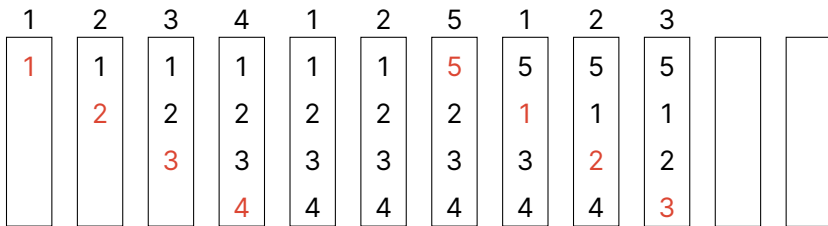
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	
1	1	1	1	1	1	5	5	5	5	4	
	2	2	2	2	2	2	1	1	1	1	
		3	3	3	3	3	3	2	2	2	
			4	4	4	4	4	4	3	3	

FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	5	5	5	5	4	4
	2	2	2	2	2	2	1	1	1	1	5
		3	3	3	3	3	3	2	2	2	2
			4	4	4	4	4	4	3	3	3

FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

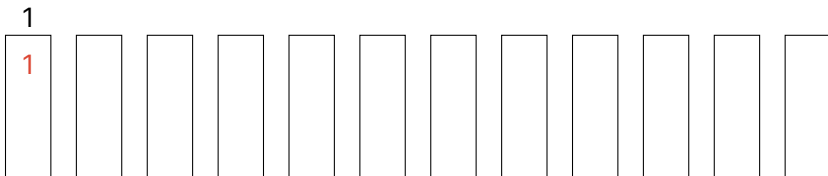
1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	5	5	5	5	4	4
	2	2	2	2	2	2	1	1	1	1	5
		3	3	3	3	3	3	2	2	2	2
			4	4	4	4	4	4	3	3	3

10 page faults

FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

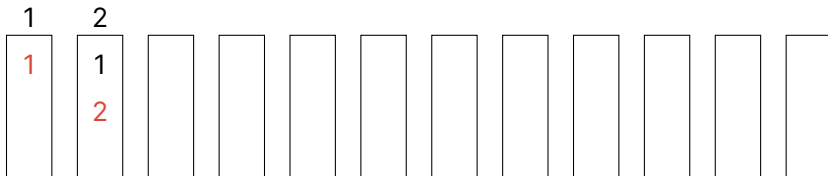
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

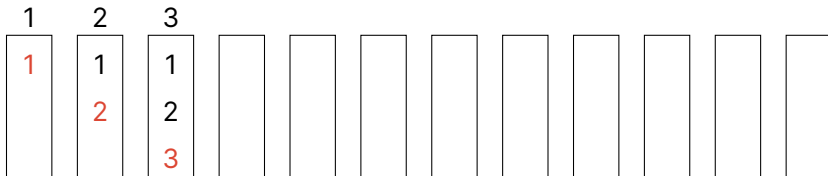
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

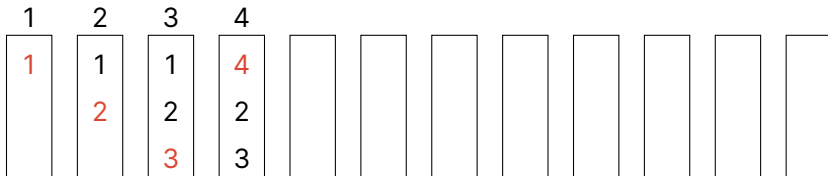
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

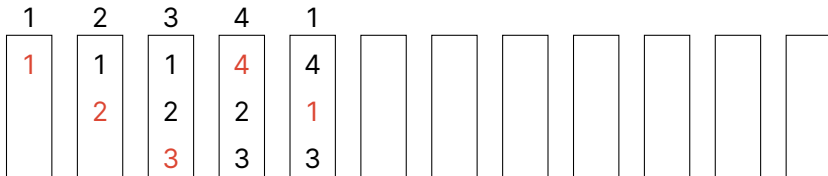
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

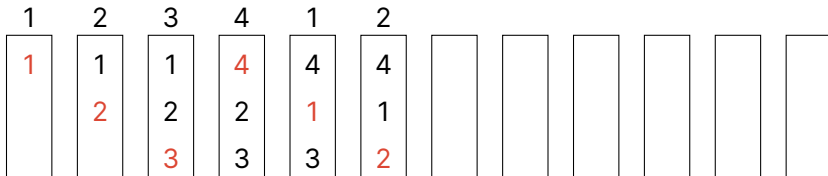
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

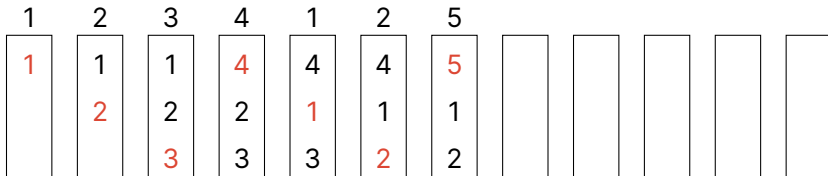
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

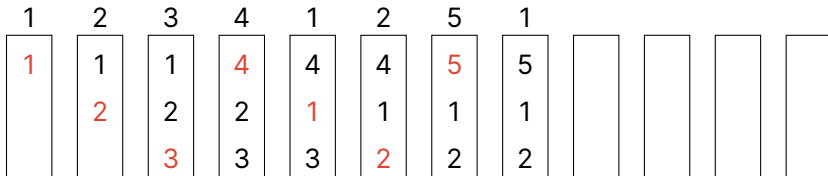
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

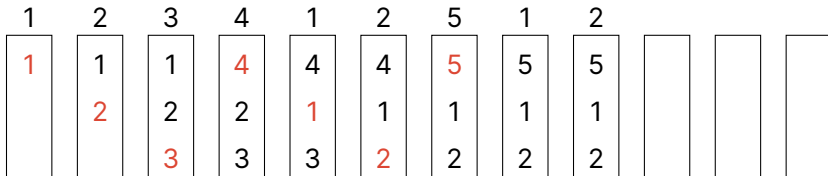
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

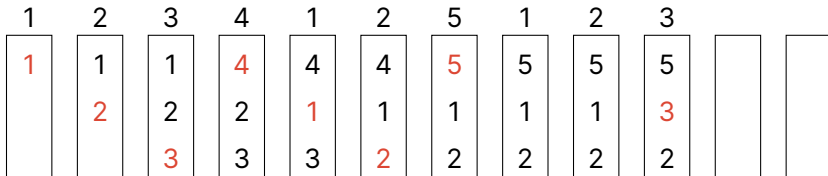
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	
1	1	1	4	4	4	5	5	5	5	5	
	2	2	2	1	1	1	1	1	3	3	
		3	3	3	2	2	2	2	2	4	

FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	4	4	4	5	5	5	5	5	5
	2	2	2	1	1	1	1	1	3	3	3
		3	3	3	2	2	2	2	2	4	4

FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	4	4	4	5	5	5	5	5	5
	2	2	2	1	1	1	1	1	3	3	3
		3	3	3	2	2	2	2	2	4	4

9 page faults

Bélády's Anomaly Says More Page Frames Causes More Faults

This is a problem with FIFO algorithms

Does not exist with LRU or “stack-based algorithms”

Paper in 2010 found that this FIFO anomaly is unbounded

(<https://arxiv.org/abs/1003.1336>)

You could construct a sequence to get any arbitrary page fault ratio

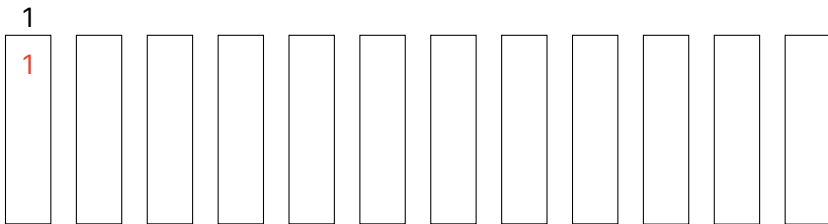
For other algorithms:

increasing the number of page frames decreases the number of page faults

LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

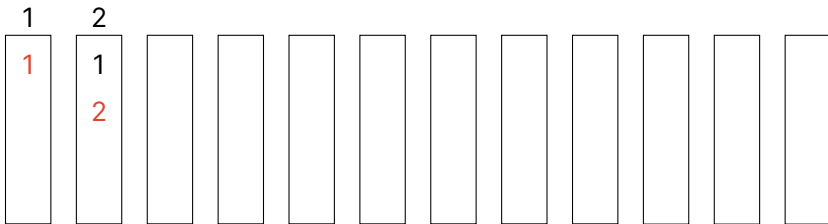
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

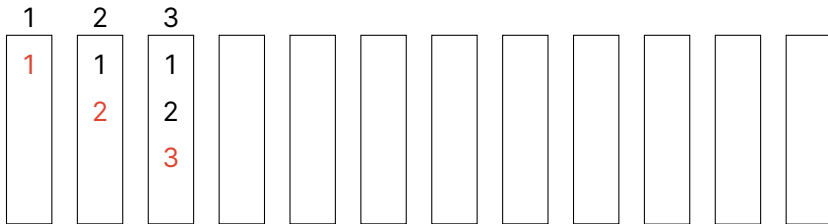
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

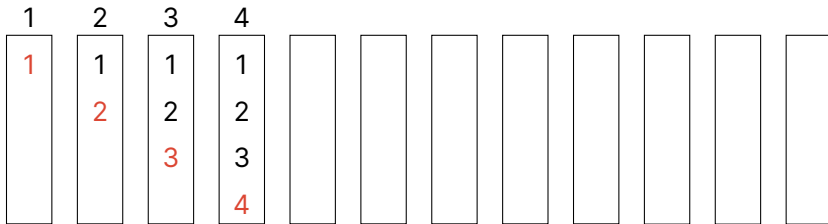
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

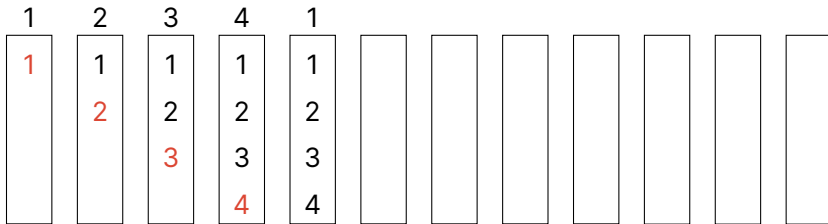
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

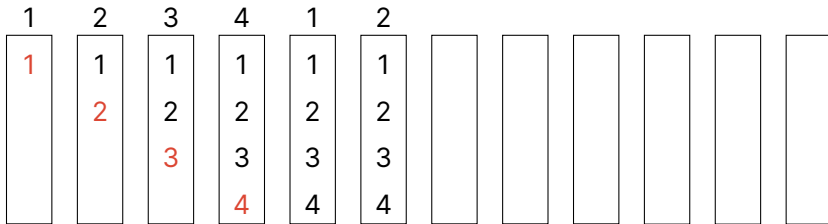
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

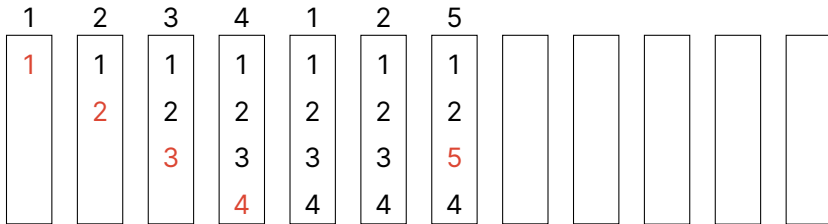
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

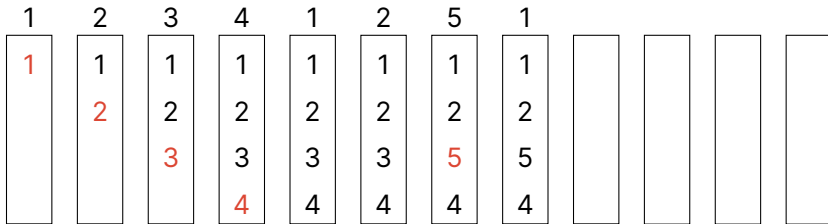
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

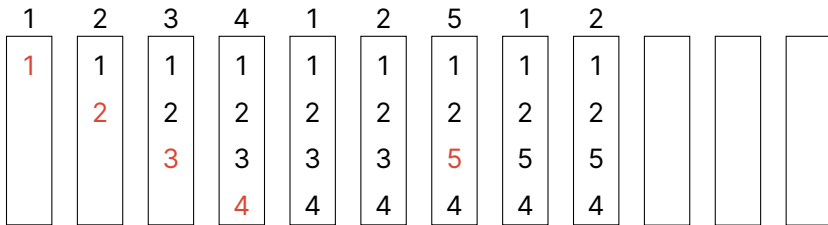
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

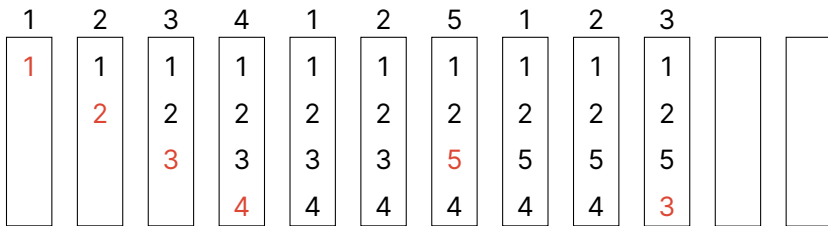
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	
1	1	1	1	1	1	1	1	1	1	1	
	2	2	2	2	2	2	2	2	2	2	
		3	3	3	3	5	5	5	5	4	
			4	4	4	4	4	4	3	3	

LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	1	5
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	5	5	5	5	4	4
			4	4	4	4	4	4	3	3	3

LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	1	5
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	5	5	5	5	4	4
			4	4	4	4	4	4	3	3	3

8 page faults

Implementing LRU in Hardware Has to Search All Pages

You could implement it by keeping a counter for each page

For each page reference, save the system clock into the counter

For replacement, scan through the pages and find the one with the oldest clock

Implementing LRU in Software is Too Expensive

- Create a doubly linked list of pages

- For each page reference, move it to the front of the list

- For replacement, remove from the back of the list

- It requires 6 pointer updates for each page reference, and
also creates a high contention bottleneck for multiple processors

Implementing LRU in Practice Isn't Going to Work

We settle for approximate LRU

- LRU is an approximation of the optimal case anyways

There's lots of different tweaks you can do to implement it more efficiently

We'll be looking at the clock algorithm, but there's also:

- Least Frequently Used (LFU), 2Q, Adaptive Replacement Cache (ARC)

Page Replacement Algorithms Aim to Reduce Page Faults

We saw the following:

- Optimal (good for comparison but not realistic)
- Random (actually works surprisingly well, avoids the worst case)
- FIFO (easy to implement but Bélády's anomaly)
- LRU (gets close to optimal but expensive to implement)

Paging Revisit

- `int A[100];`
- All instructions are on page 0.
- Each page holds 64 entries (64 integers).
- Array A starts at location 64.

Page Replacement Algorithm

Memory can hold only a limited number of pages.

When memory is full, a page must be swapped out to swap space (where?).

The OS decides which page to replace using a Page Replacement Algorithm.

Goal: Keep the most useful pages in memory to improve performance (reduce the number of page faults).

Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

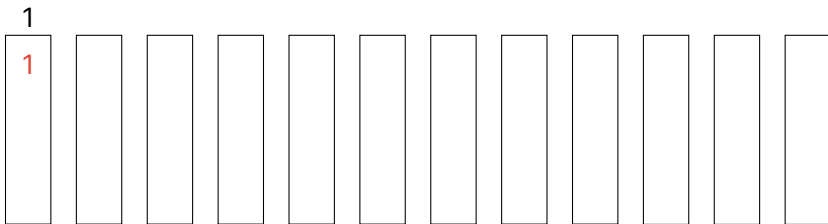
1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	4	4
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	3	3	3	3	3	3
			4	4	4	5	5	5	5	5	5

6 page faults

FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

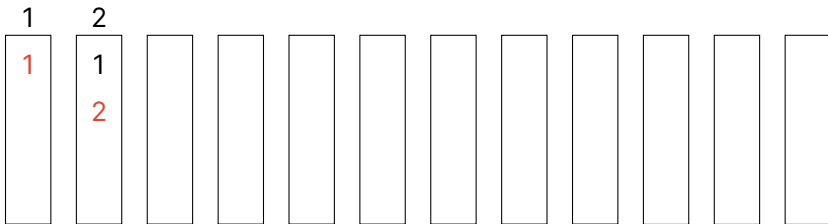
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

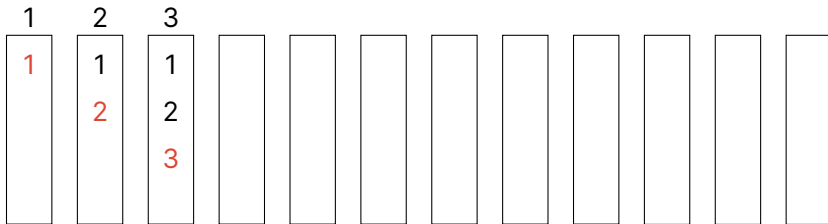
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

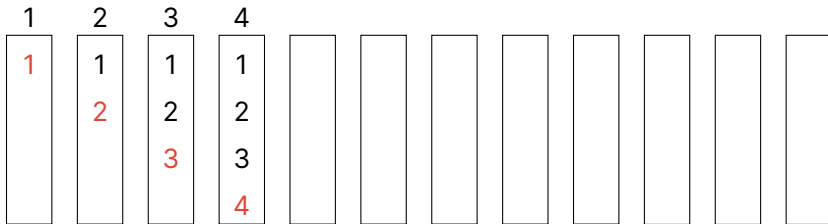
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

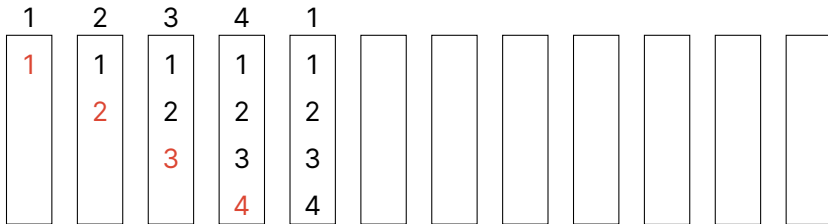
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

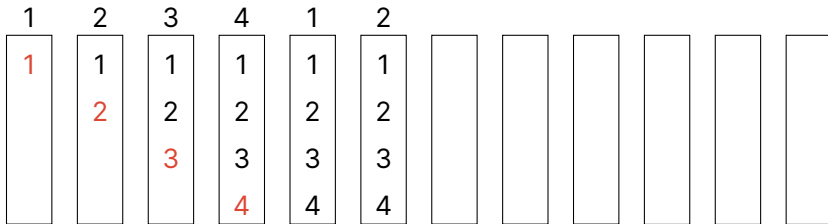
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

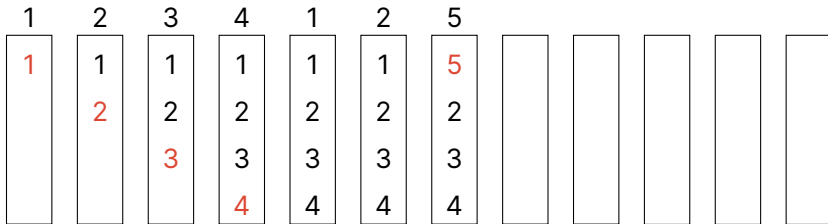
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

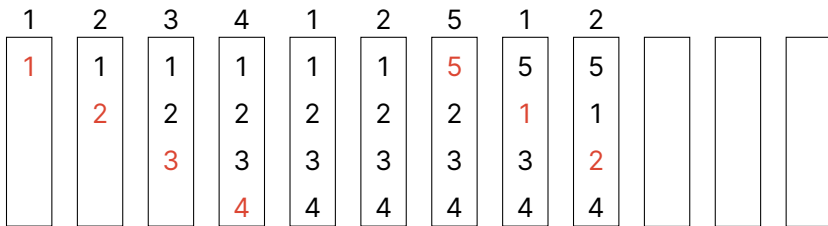
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

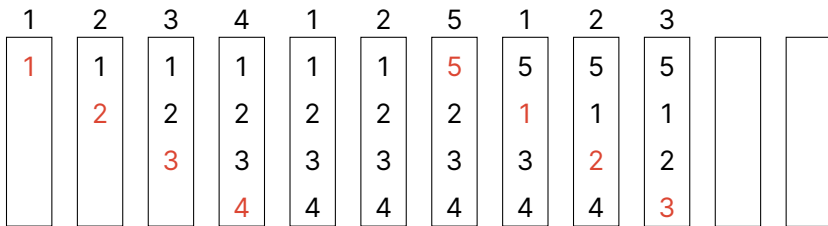
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	
1	1	1	1	1	1	5	5	5	5	4	
	2	2	2	2	2	2	1	1	1	1	
		3	3	3	3	3	3	2	2	2	
			4	4	4	4	4	4	3	3	

FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	5	5	5	5	4	4
	2	2	2	2	2	2	1	1	1	1	5
		3	3	3	3	3	3	2	2	2	2
			4	4	4	4	4	4	3	3	3

FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

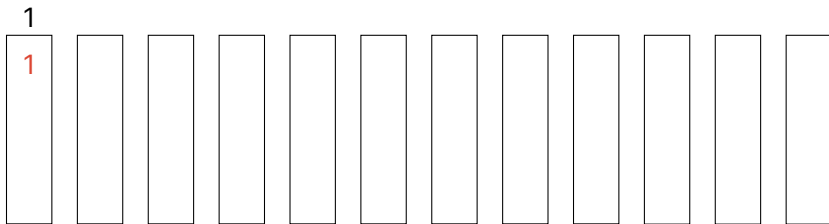
1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	5	5	5	5	4	4
	2	2	2	2	2	2	1	1	1	1	5
		3	3	3	3	3	3	2	2	2	2
			4	4	4	4	4	4	3	3	3

10 page faults

LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

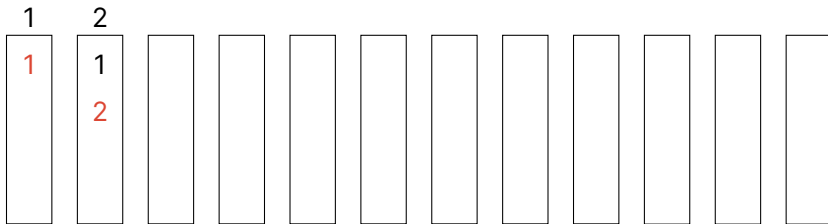
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

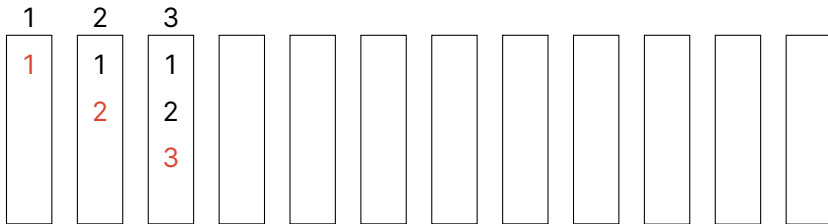
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

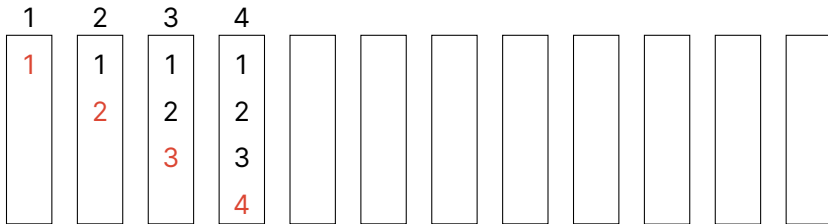
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

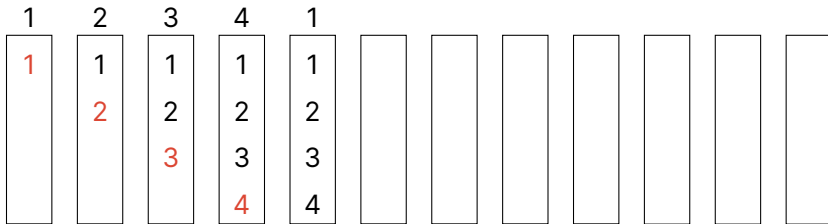
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

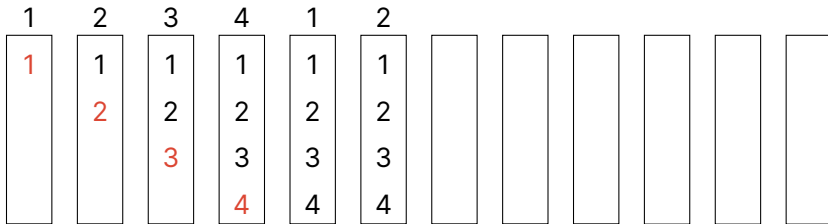
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

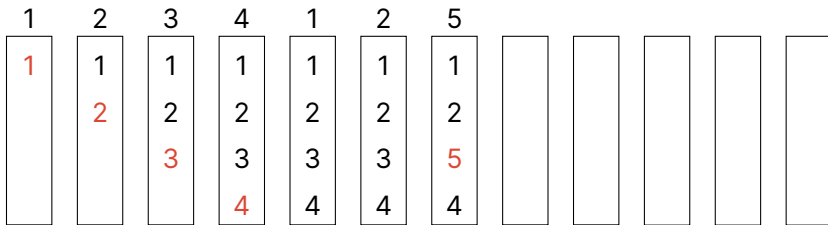
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

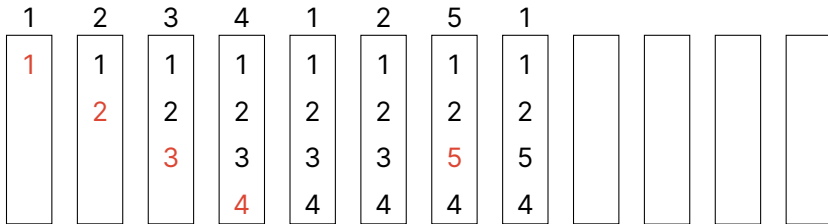
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

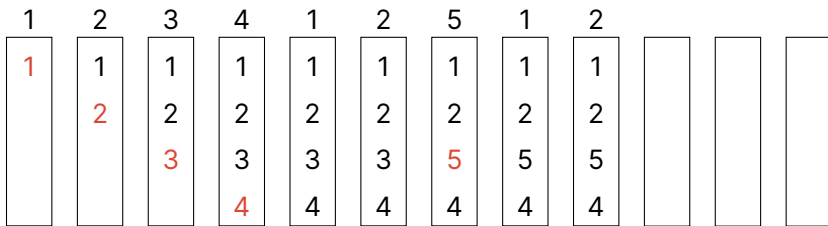
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

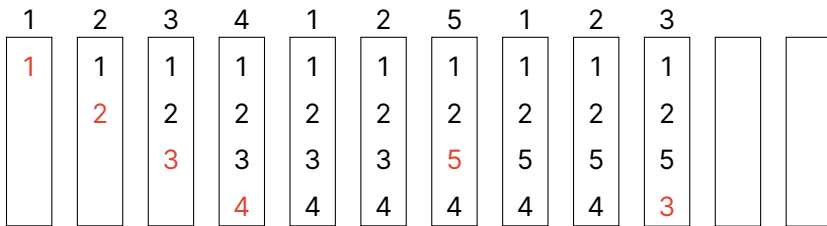
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	
1	1	1	1	1	1	1	1	1	1	1	
	2	2	2	2	2	2	2	2	2	2	
		3	3	3	3	5	5	5	5	4	
			4	4	4	4	4	4	3	3	

LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	1	5
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	5	5	5	5	4	4
			4	4	4	4	4	4	3	3	3

LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	1	5
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	5	5	5	5	4	4
			4	4	4	4	4	4	3	3	3

8 page faults

Implementing LRU in Practice Isn't Going to Work

We settle for approximate LRU

- LRU is an approximation of the optimal case anyways

There's lots of different tweaks you can do to implement it more efficiently

We'll be looking at the clock algorithm, but there's also:

- Least Frequently Used (LFU), 2Q, Adaptive Replacement Cache (ARC)

Clock Algorithm

Data structures:

- Keeps a circular list of pages in memory
- Uses a reference bit for each page in memory (light grey in next slides)
- Has a “hand” (iterator) pointing to the last element examined

Algorithm, to insert a new page:

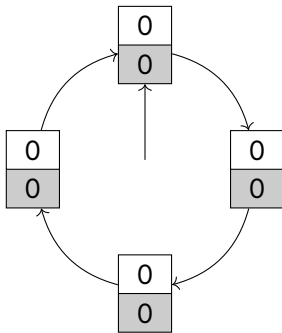
- Check the hand's reference bit, if it's 0 then place the page and advance hand
- If the reference bit is 1, set it to 0, advance the hand, and repeat

For page accesses, set the reference bit to 1

Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

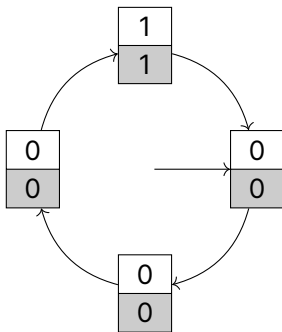


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1

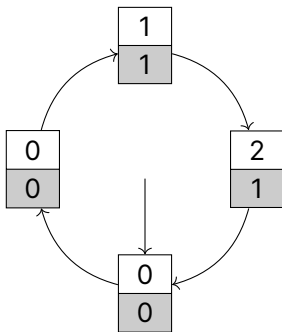


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2

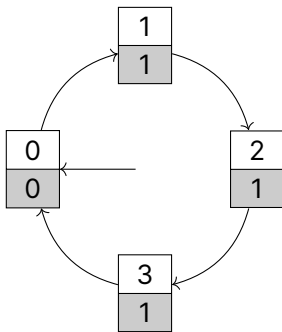


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3

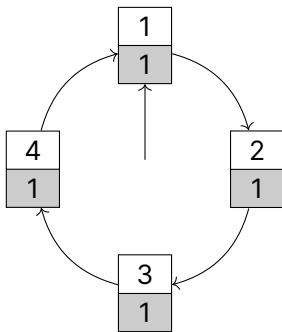


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4

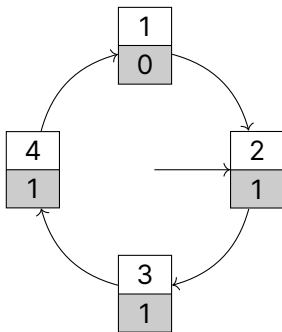


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5

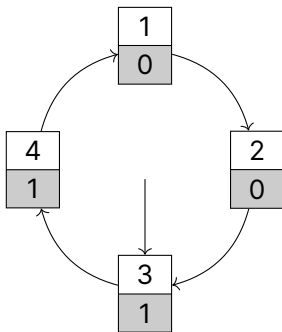


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5

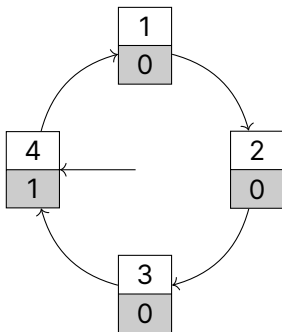


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5

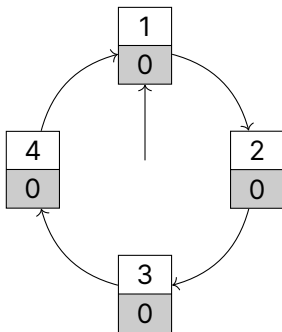


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5

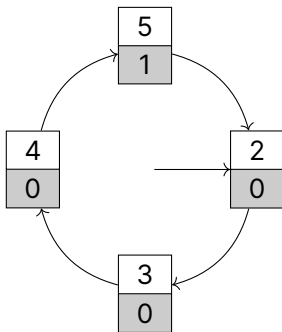


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5

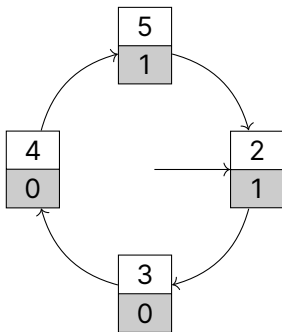


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2

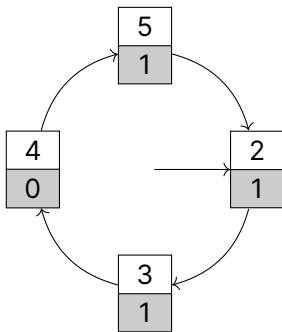


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3

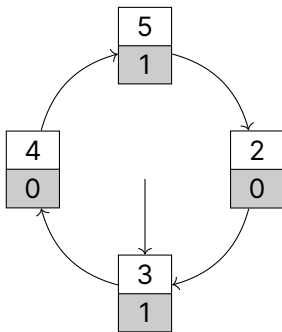


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3 1

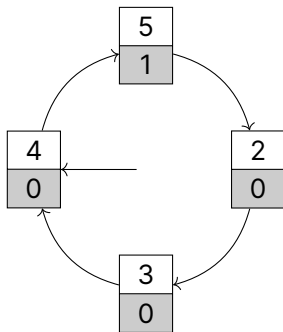


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3 1

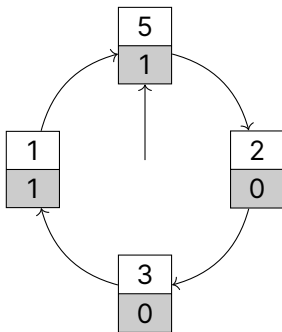


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3 1

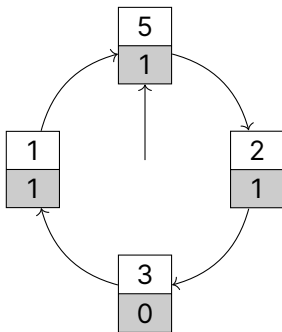


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3 1 2

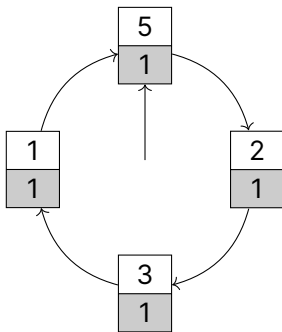


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3 1 2 3



Clock Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1	2	3	4	5	2	3	1	2	3
1	1	1	1	5	5	5	5	5	5
	2	2	2	2	2	2	2	2	2
		3	3	3	3	3	3	3	3
			4	4	4	4	1	1	1

6 page faults

The Buddy Allocator Restricts the Problem

Typically, allocation requests are of size 2^n

e.g. 2, 4, 8, 16, 32, ..., 4096, ...

Restrict allocations to be powers of 2 to enable a more efficient implementation

Split blocks into 2 until you can handle the request

We want to be able to do fast searching and merging

You Can Implement the Buddy Allocator Using Multiple Lists

We restrict the requests to be 2^k , $0 \leq k \leq N$ (round up if needed)

Our implementation would use $N + 1$ free lists of blocks for each size

For a request of size 2^k , search the free list until we find a big enough block

- Search $k, k + 1, k + 2, \dots$ until we find one

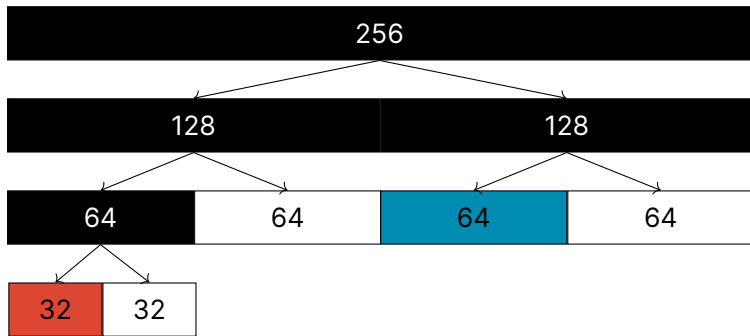
- Recursively divide the block if needed until it's the correct size

- Insert “buddy” blocks into free lists

For deallocations, we coalesce the buddy blocks back together

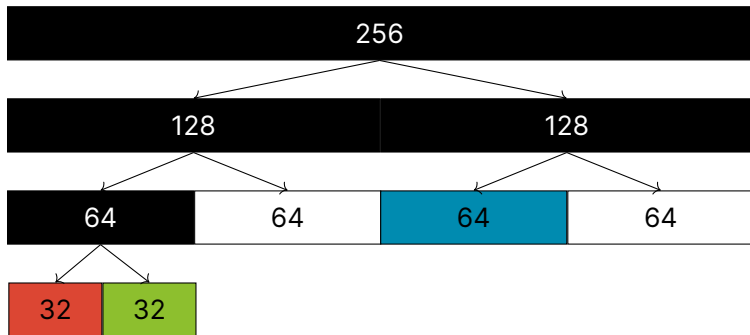
- Recursively coalesce the blocks if needed

Using the Buddy Allocator (1)



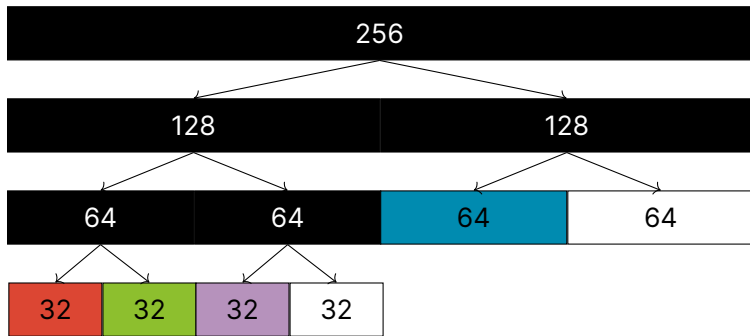
Where do we allocate a request of size 28?

Using the Buddy Allocator (2)



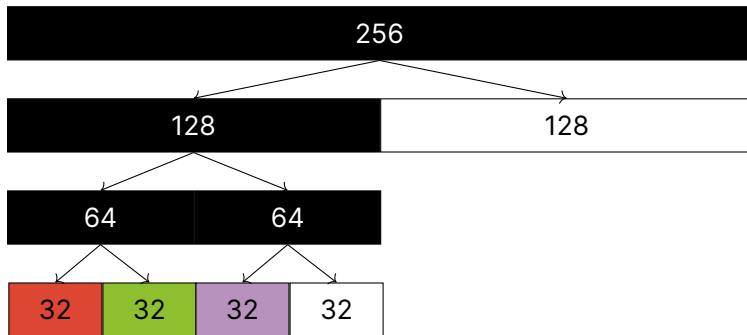
Where do we allocate a request of size 32?

Using the Buddy Allocator (3)



What happens when we free the size 64 block?

Using the Buddy Allocator (4)



Buddy Allocators are Used in Linux

Advantages

- Fast and simple compared to general dynamic memory allocation
- Avoids external fragmentation by keeping free physical pages contiguous

Disadvantages

- There's always internal fragmentation
- We always round up the allocation size if it's not a power of 2

Slab Allocators Take Advantage of Fixed Size Allocations

- Allocate objects of same size from a dedicated pool

 - All structures of the same type are the same size

- Every object type has it's own pool with blocks of the correct size

 - This prevents internal fragmentation

Slab is a Cache of “Slots”

Each allocation size has a corresponding slab of slots
(one slot holds one allocation)

Instead of a linked list, we can use a bitmap
(there's a mapping between bit and slot)

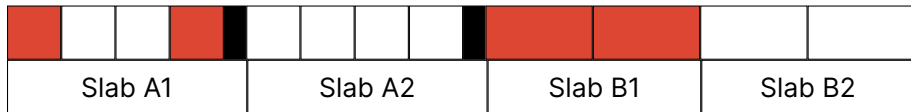
- For allocations, we set the bit and return the slot

- For deallocations, we just clear the bit

The slab can be implemented on top of the buddy allocator

Each Slab Can Be Allocated using the Buddy Allocator

Consider two object sizes: A and B



We can reduce internal fragmentation if Slabs are located adjacently
In this example A has internal fragmentation (dark box)

Journaling Filesystem

Deleting a file on a Unix file system involves three steps:

1. Removing its directory entry.
2. Releasing the inode to the pool of free inodes.
3. Returning all disk blocks to the pool of free disk blocks.

Crashes could occur between any steps, leading to a storage leak

The journal contains operations in progress, so if a crash occurs we can recover